

Number Guessing Game – Final Project Report - Anshuman Singh

1. Project Description

A small number guessing challenge runs inside the command line, built using Python. The objective is a hidden random value must be found by the user through trial. Following every try, a message shows if the input sits above or below the target. That hint shapes what comes next - each round narrows the search. Victory arrives when the answer clicks - or chances fade, based on selected strictness.

One feature built into the system lets players choose how hard the game should be. A counter follows each try someone makes at solving a level. Data stays saved between sessions through stored records tied to users. Looking back at earlier rounds is possible thanks to logs kept over time. Each addition supports smoother interaction while using the software. Most of these elements show practical uses of fundamental coding ideas that go further than what was strictly needed.

2. Approach and Logic Used

A structure built in modules separates tasks across unique functions. Because each piece manages its own role, understanding grows simpler. When updates occur, only specific sections need attention - this makes changes easier to manage. Errors show up clearly since components work independently. Through separation, fixing problems becomes more direct.

Beginning the program shows a menu offering options: playing, checking previous results, or leaving. Should the person pick gameplay, a request appears for their name followed by choosing how hard the challenge will be. Difficulty shapes both the span of the generated number and the tries available.

An integer emerges at random through Python's `random.randint()` function. Following that, the software begins cycling - asking the person to submit estimates again and again. Each time one of those attempts arrives:

- If the guess is lower than the number, the program outputs "Too low."
- If the guess is higher, it outputs "Too high."
- If the guess is correct, the game ends with a success message.

A safeguard against crashes is through the use of a try-except structure whenever input fails verification. Progress persists within the loop unless either the right answer appears or attempts run out.

Once play finishes, the outcome - whether victory or defeat - is saved alongside attempt count plus challenge setting within a document meant for later checking.

3. Data Structures Used

Various ways of organising information work well here

- **Lists and Dictionaries:**

Looking up players? The system sorts info using a dictionary keyed by name. One name, one entry - each holds a collection of that person's game history. Grouping them this way makes it easier to track repeated plays. Instead of scattered details, everything ties back to the individual. Finding patterns becomes simpler when entries stack neatly under each tag.

- **Class (Object-Oriented Programming):**

One way to hold details about a single game round is through a GameRecord class. Holding pieces like who played, how many tries they made, the challenge setting, along with win or loss comes next. Better structure shows up once these parts get grouped into objects. Using ideas from object-based design appears naturally when building it this way.

- **File Handling:**

A text file named records.txt holds the game history. One game per line appears inside it. Data stays saved between uses because of this setup.

4. Challenges Faced

Handling wrong entries turned into a real puzzle. When people typed strange stuff - letters instead of numbers, wild guesses off the scale - the system had to stay on its feet, not break apart. Each odd input needed quiet correction, smooth recovery.

Figuring out how to save the records brought up a new problem. At first, every entry was just one line in a file, sitting on its own. Getting these to show up neatly meant sorting them based on who played. A shift happened when a list inside a dictionary took care of that job. Afterward, things improved once each record got its own class setup.

Smooth running of programs, especially when using menus and repeated sections, meant each part had to fit just right - otherwise some parts would never run or others would repeat forever. Pacing the steps helped keep everything moving without getting stuck somewhere by mistake.

5. Possible Improvements

Though the system works as expected now - even with extra features built in - different upgrades remain possible

- A window pops up, built with tools like Tkinter. This makes clicking through tasks smoother than typing commands. Through visual buttons, people navigate without memorizing steps. Instead of plain text prompts, shapes and menus guide each move. Interaction feels familiar, much like other everyday software. Design shifts toward how eyes track space on screen. Each element sits where hands expect it. Clicking replaces long keyboard strings. People see options instead of guessing them. Smooth paths form between start and finish.
- Keeping information in JSON helps organize it better. That way working with data feels less messy. A clearer layout means handling details gets simpler.
- Add a feature to display the best score (minimum attempts) for each player.
- A fresh scoreboard shows who's ahead. Names climb higher when points rise. Standing shifts each time someone wins. Rankings update right after every match. Top spots go only to those scoring most.
- Try adding clues like "really near" to get players more involved.

Built tougher, the app would handle tasks better while feeling easier to use. Suddenly, small changes could lead to smoother experiences without slowing things down.

Conclusion

A handful of core coding ideas come into play here - loops appear alongside conditionals, while functions team up with file operations and a touch of object-style design. Built with clear organization, the software manages unusual inputs without stumbling, keeping things smooth for whoever uses it. Extra layers like adjustable challenge settings along with saved results lift the whole thing higher, turning it from simple code into something that feels finished and ready to go.